

ABRA — Architecture Review

The scathing review, the eleven standards that follow from it, and what is enforced by what.

Will Hooper · 2026-07-24

Read this as one piece. Section 1 is the list of faults this project actually shipped. Section 2 turns each fault into a rule with a literature basis. Section 4 is the table that says which check enforces which rule - a standard with no check in that column is only a preference. The changelog entries at the end record the two releases this review produced.

Contents

1. Architecture and Engineering Standards
2. Architecture notes (languages, server, recency)
3. Changelog

ABRA — System Architecture and Engineering Standards

Version 1.1 · 2026-07-24 · Will Hooper

A review of the whole system, the rules that follow from it, and how those rules are enforced. This document is deliberately blunt. Every fault named here is one this project actually shipped, and each is paired with the standard that prevents its recurrence and the check that enforces it.

1. The review, without softening

The models are careful. **The plumbing was not.** Every serious defect found on 2026-07-24 was an infrastructure fault, not a modelling error — and each one silently corrupted a model that was otherwise sound. In order of severity:

1.1 Three implementations of the same rules. Damage, stat stages, spread and mega handling were implemented in `CHOMP/engine/champ-model.js`, `ABRA/engine/medicham2-browser.js`, and a third embedded copy in `ABRA/web/index.html`. Fowler's *Rule of Three* says the third duplication is the moment to act; we were already past it. The cost was realised: when the canonical engine was taught real mega base stats, the rollout engine was left behind and the two disagreed by **30%** on Charizard-Mega-Y's Special Attack, with nothing failing.

1.2 Two hand-maintained copies of the same table, from different sources. Mega abilities existed in `champ-model.js` (from Showdown) and in `medicham2-browser.js` (from Serebii). They happened to agree. Nothing guaranteed it, and nothing would have reported it if they had not.

1.3 A data file that stored only derived values. `data/engine-data.js` held computed level-50 stat lines but no base stats. Derived-only data **cannot be recomputed** — so when the forme rules changed, the browser engine could not follow. It could only copy a number that was already wrong.

1.4 A fix applied to the wrong artifact. 67 mega formes were merged into `engine-data.js` and reported as "in the engine dex". The damage engine reads a *different* file (`champions-damage-lab.html`), parsed with `eval()`, so the fix reached nothing. The claim was made in good faith and was false.

1.5 A parser change that silently destroyed an eval set. Recording mega evolution added the mega forme to `brought`, so a Pokémon that mega-evolved counted twice. `brought` became 5 in ~4,700

games, which failed CHOMP-EV's "exactly four" filter and collapsed its eval set from ~1,200 games to **43**. No test guarded the shape of the store.

1.6 Half the store was duplicate rows. 14,361 lines, 7,315 unique games — 7,040 duplicates from merge -X ours reconciliations. Every "14,355 games" claim was inflated ~2×, and duplicated rows narrow confidence intervals without adding information. Two marginal results lost significance when it was fixed.

1.7 Cross-boundary identifiers were not normalised. "sand stream" in one engine, "sandstream" in the other. Both internally consistent; a raw string compare between them is a latent bug.

1.8 Hand-typed constants presented as findings. Role weights (0.6, 0.4) and a belief boost (3.0) were invented, not measured. When measured, the boost was 1.26 and the reported gain shrank.

1.9 A lookup table that was silently incomplete. The nature table held 23 of the 25 natures. Naughty and Lax were simply absent, and an absent nature falls through to the neutral multiplier — so those sets computed with 1.0 where the game applies 1.1 and 0.9. Nothing crashed and no count looked wrong, because **a missing row and a legitimately neutral row are indistinguishable by count**. The earlier check asked "did any nature produce an unexpected multiplier?" and 1.0 is an expected multiplier, so it passed. A partial table is the same class of fault as a duplicated one: knowledge that is asserted rather than enumerated against its source.

The pattern is singular: **knowledge with more than one home, and no mechanism that notices when the homes disagree** — or, in 1.9, knowledge with one home that was never checked against the source it claims to represent.

2. The standards

S1 — Single source of truth, and generate the rest

"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system." — Hunt & Thomas, The Pragmatic Programmer (1999)

Where multiple representations are unavoidable (a browser copy, a cached table), **one is definitive and every other is generated by a script**, never hand-synchronised. A generated file carries a header saying so and naming its generator.

Applied: `data/mega-formes.json`, `data/move-effects.json`, `data/battle-formes.json`, `data/species-abilities.json` and the `MC` object in `data/engine-data.js` are all generated.

S2 — Duplication that cannot be removed must be observable

Where a second implementation is genuinely required (the site is buildless and cannot `require()`), a **consumer-driven contract test** asserts that all implementations agree on the rules. This is the Pact pattern: a shared, executable statement of the agreement, run in CI, so divergence fails a build instead of corrupting a model.

Applied: `CHOMP/tests/test-engine-contract.js` — 20 assertions. It caught the mega drift on its first run.

S3 — Store source, not just derived values

Any dataset a consumer might need to recompute from must carry its **inputs**, not only its outputs. Derived values may be included for convenience; they may never be the only representation.

Applied: `engine-data.js` now carries base stats (`bs`) alongside the level-50 line (`st`).

S4 — Every dataset records its provenance

Source URL, capture date, authority level, and refresh policy. If two sources disagree, the higher-authority source wins and the disagreement is reported, not silently resolved. This follows standard practice for reproducibility in production ML (versioning, provenance, reproducibility).

Applied: `CHOMP/data/PROVENANCE.md`; every generated file carries `source` and `generated` fields.

S5 — Normalise identifiers at every boundary

Any identifier crossing a subsystem boundary is compared and keyed as **lowercase alphanumeric only**. Display forms are for humans and never for lookups.

S6 — Assert the invariant, not the incidental

A test states the claim being made. CHOMP-EV's claim is "bring quality does not separate from a coin" — a statement about an *interval*, not a point estimate. Pinning the point broke the test when the eval set legitimately changed. Assert `CI contains the coin`.

S7 — The store has a shape, and it is tested

Structural invariants are checked on every run: no duplicate ids, `brought ≤ six`, `lead ≤ brought`, the winner is one of the two players, and every record carries every field. A parser change that breaks the shape must fail immediately, not months later inside a model.

S8 — Measured, never asserted

No constant that affects a result is typed by hand. If a weight matters, it is estimated from training data and the estimate is reported. If it cannot be estimated, the feature is not shipped.

S9 — Golden-master before refactoring

Before changing a computation, record its outputs; after, compare. Approval testing exists precisely to make behaviour-preserving refactors provable rather than hoped for.

Applied: `engine/validate_damage.js` (31 scenarios vs the Smogon calculator) is run before and after every engine change.

S10 — Enumerate the domain, do not spot-check it

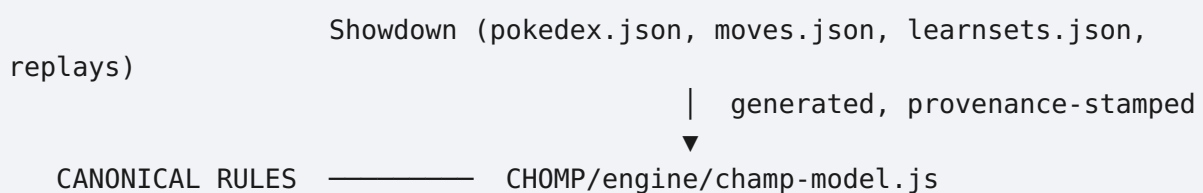
Where a rule has a **closed, known domain** — 25 natures, 18 types, 6 stat stages — the test iterates the whole domain and asserts the expected behaviour for each member, including a count assertion that the reference list is complete. Spot-checking a table cannot detect a missing row, because absence usually degrades to a plausible default rather than an error.

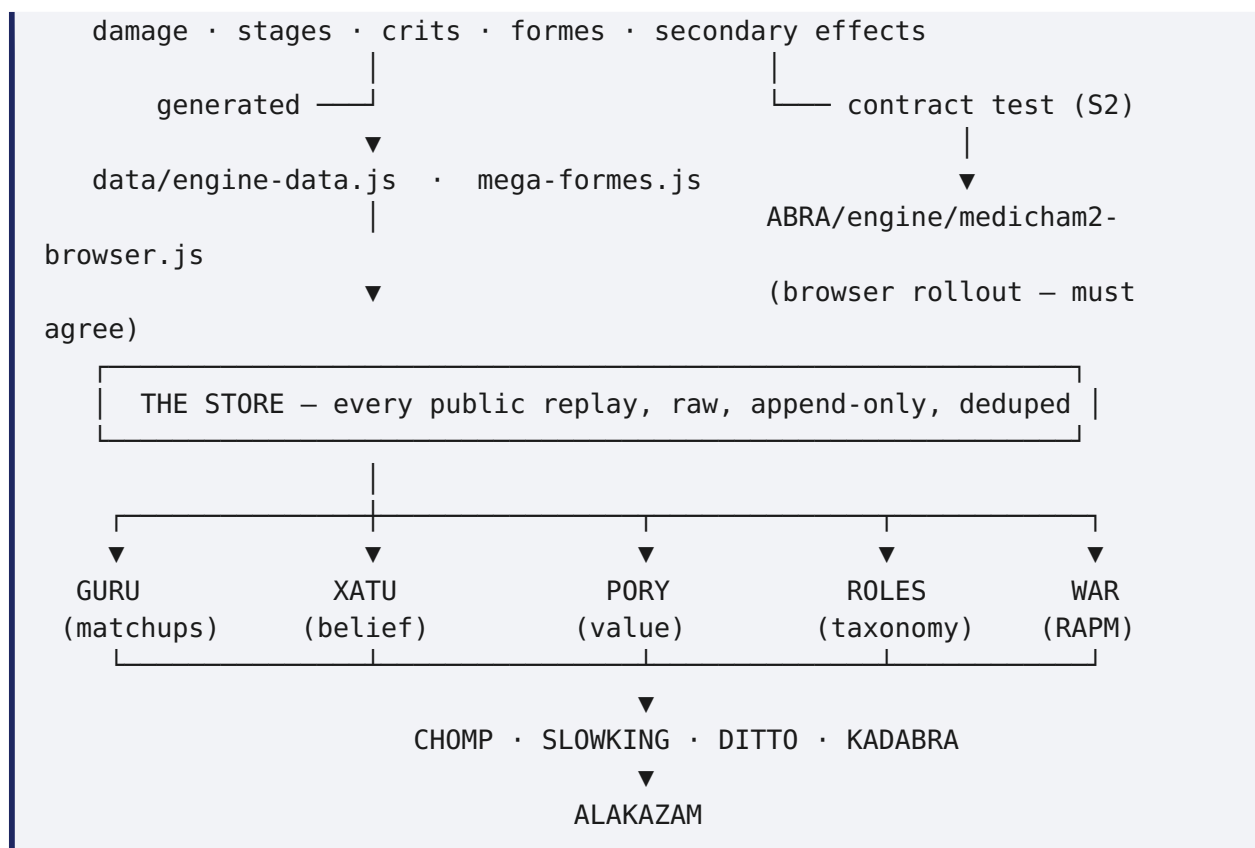
Applied: `test-mega-and-boosts.js` now walks all 25 natures and asserts the **direction** of change for all five stats against a neutral baseline. It found Naughty and Lax on its first run.

S11 — One publisher

Exactly one process commits and pushes. Two publishers racing produced a wedged rebase and hours of lost work. Any automation touching git refuses to act on a repository that is mid-rebase or mid-merge.

3. The intended shape





Only one component needs a damage calculator. Everything below it consumes the result. The statistical models (GURU, XATU, PORY, ROLES, WAR) never touch damage at all and are unaffected by engine changes — which is why none of the faults above reached them.

4. Enforcement

Standard	Enforced by	Status
S1 generate, don't synchronise	generator scripts + <code>generated</code> / <code>source</code> headers	applied
S2 contract test	<code>CHOMP/tests/test-engine-contract.js</code>	20 assertions, passing
S3 store source values	<code>build_engine_data.js</code> emits <code>bs</code>	applied
S4 provenance	<code>CHOMP/data/PROVENANCE.md</code> + per-file fields	applied
S5 identifier normalisation	contract test compares normalised	applied
S6 assert the invariant	<code>sanity_check.py</code> CHOMP-EV interval test	applied

Standard	Enforced by	Status
S7 store shape	<code>sanity_check.py</code> §6 + integrity audit	applied
S8 measured not asserted	<code>test-xatu-belief.py</code> re-derives the boost	applied
S9 golden master	<code>validate_damage.js</code> before/after	applied
S10 enumerate closed domains	<code>test-mega-and-boosts.js</code> walks all 25 natures	applied
S11 one publisher	<code>push-all.bat</code> guard, mid-rebase refusal	applied

5. Known gaps, stated plainly

- **The canonical engine's dex is still scraped from an HTML file** (`champions-damage-lab.html` , parsed with `eval()`). It should be a real data file with a generator. This is the single largest remaining violation of S1 and is the reason fault 1.4 was possible.
- **The site's embedded engine copy** (`web/index.html`) is not yet covered by the contract test.
- **Secondary effects are defined but not yet consumed by the rollout engine** — the rulebook exists and is tested at the point of definition; wiring it into turn simulation is outstanding.
- **brought** is still 5 in ~115 games and 0 in 176; not yet explained.

Sources

Hunt & Thomas, *The Pragmatic Programmer* (1999) — DRY, single source of truth · Fowler, *Refactoring* — Rule of Three, duplicated logic · consumer-driven contract testing (Pact) · approval / golden-master testing · versioning, provenance and reproducibility in production ML (Kästner, CMU).

ABRA — Architecture Notes

Version 1.0 · Last updated 2026-07-23 · Will Hooper

The design decisions behind ABRA that aren't obvious from any single file: why the models are split across two languages, how the local app runs the real engines on your own machine, and how recency weighting keeps every model tracking the live metagame. Living document; updated with the code.

1. Why two languages (Python and JavaScript)

ABRA's engines are deliberately split. It is not an accident or inconsistency — each half of the system lives in the language that makes it shortest and safest to write.

The rule

- **JavaScript (Node) for anything that touches the battle engine.** CHOMP's damage calculator (`CHOMP/engine/champ-model.js`) is the validated, exact model of Champions damage — types, STAB, weather, items, abilities, the full 16-roll distribution. It is written in JS. Every model that needs *real game mechanics* reuses it directly instead of re-implementing (and re-validating) the damage formula:
- **MEDICHAM** (`engine/medicham.js`) — rollouts call CHOMP's `bestDamage` per hit.
- **KADABRA** (`engine/kadabra.js`) — the coach reconstructs scenes and cross-checks damage.
- **the set-builder, behaviour-clone, dynamics** — all Node, all sharing the same move/typing data. Re-porting the damage engine to Python would mean maintaining two copies of the most correctness- critical code in the project. That is the thing we most want to avoid.
- **Python (with numpy) for the statistical / machine-learning core.** JOLTEON's training (`engine/jolteon.py`) and DITTO's search (`engine/ditto.py`) are linear algebra: logistic-regression gradient descent, vectorised scoring of thousands of teams, rarity-aware regularisation, recency weighting. numpy expresses this in a few lines and runs it fast. Writing weighted logistic regression by hand in JS would be longer and slower with no upside.

The boundary, and why it barely matters to a user

The split falls on a clean seam: **mechanics = JS, learned numeric models = Python**. The one place they meet is DITTO, which uses JOLTEON (Python) as its fast evaluator and MEDICHAM (JS) as its grounded vetter — it shells out to both.

Crucially, **using the models needs almost no Python**: - JOLTEON's *predictions* are a tiny logistic dot-product. We ship the trained weights as JSON and recompute the probability in pure JavaScript in the browser (`web/index.html`) and could do so in Node. You never need Python to *use* JOLTEON — only to *retrain* it. - MEDICHAM, KADABRA, CHOMP are Node end to end. - Only **DITTO** currently requires Python at run time (numpy for the team search). Because that is the sole run-time Python dependency, the planned **Node port of DITTO** removes Python from the live app entirely: JOLTEON weights are already portable, MEDICHAM is already JS, so a JS DITTO closes the loop and the whole `localhost` app runs on Node alone. Retraining JOLTEON stays in Python, offline, where numpy earns its keep.

The Windows Python gotcha (documented because it will bite)

Windows ships a fake `python` command that opens the Microsoft Store instead of running Python. So on a machine without a real Python install, `python engine/ditto.py` prints "*Python was not found; run without arguments to install from the Microsoft Store.*" The server (`server.js`) handles this: it probes `py`, then `python`, then `python3`, skips the Store stub, checks that numpy imports, and — if no real Python is found — returns a clear message telling you JOLTEON/MEDICHAM/KADABRA still work and how to install Python for DITTO (python.org, "Add to PATH", then `pip install numpy`). This probe is **lazy** (run on the first DITTO/JOLTEON request, not at startup) so the server always starts listening instantly.

2. The local app — running the real engines on your machine

`web/index.html` (mirrored to `app/index.html`, which the server prefers) is the ABRA WORLD site. Opened as a file, it runs **JOLTEON** live in the browser (embedded weights) — that alone needs no server. To run the *other* engines, `start.bat` launches `server.js`, a dependency-free Node HTTP server that: - serves the site from `app/` (falling back to `web/`), - exposes `/api/medicham`, `/api/kadabra` (Node engines), `/api/jolteon`, `/api/ditto` (Python engines), and `/api/stats` (live counts for the town), - runs every computation **on your own machine** — nothing is sent anywhere.

`start.bat` checks Node is installed, opens the browser *after* a short delay (so the first load doesn't race the server), and keeps a window open as the running server. Everything is local; "server" here just means "a small program using your computer's own CPU," not a remote host.

3. Recency weighting — tracking a metagame that moves

The problem: concept drift

A competitive metagame is **non-stationary**. The teams that win shift week to week as players adapt, new cores emerge, and old ones fall off. A model trained with every game counted equally averages over that history, so it is always partly describing a metagame that no longer exists. In machine-learning terms this is **concept drift**, and the standard remedy is to weight recent data more heavily.

The mechanism: exponential decay with a half-life

Every training game gets a weight that decays exponentially with its age:

$$w_i = 0.5^{(age_days_i / \tau)}$$

where `age_days_i` is how old game *i* is relative to the newest game in the set, and **τ (tau) is the half-life in days**. Interpretation: - a game from *today* has weight ~ 1 , - a game **one half-life** old counts **half**, - **two half-lives** $\rightarrow$ a quarter, **three** $\rightarrow$ an eighth, and so on.

Weights are normalised to mean 1 so the regularisation scale (the rarity-aware L2) is unchanged; only the *relative* influence of recent vs. old games shifts. Setting $\tau \rightarrow \infty$ recovers the old equal-weight behaviour exactly, so decay is a strict generalisation, not a different model.

Where it plugs in

The weights enter the **weighted logistic-regression gradient** in JOLTEON's `fit()`:

$$\text{grad} = X^T (w \odot (p - y)) / \Sigma w + L2 \cdot \theta / \Sigma w$$

The same `$w_i = 0.5^{(age/\tau)}$` applies identically to the **usage model** (weighted species counts) and the **behaviour-clone** (weighted move frequencies), so JOLTEON, DITTO's meta gauntlet, CHOMP's usage intel, and MEDICHAM's move priors all track the *current* ladder from one shared decay rule. The default half-life is **30 days** (`HALF_LIFE` env var); shorter τ reacts faster to a shifting meta but uses effectively less data, longer τ is steadier but laggier — a bias/variance knob.

Honest status on current data

ABRA's store currently spans ~ 2 days, so at $\tau = 30$ days every game's weight is ~ 1 and decay is a **no-op** — reported honestly (the equal-weight and recency-decayed rows are identical). The mechanism is unit-tested on synthetic 90-day data (oldest game weight 0.33, newest 2.14 at $\tau = 30$ d) and will

engage automatically as the corpus ages past a few weeks. It costs nothing now and means the models never need manual pruning of stale games later — the influence just fades on its own.

Why not simply delete old games?

Because rare-but-still-relevant information (an off-meta answer that occasionally matters) survives in old games, and a hard cutoff throws it away discontinuously. Exponential decay is a soft, continuous down-weighting: old games keep a small vote, recent games dominate, and nothing is ever lost from the store (which stays the durable, replayable record).

Related. [Simulator white paper](#) · [SLOWKING white paper](#) · [Cheat sheet](#) · [Changelog](#)

PART 3

Changelog — ABRA

All notable changes to ABRA are recorded here, newest first. The format follows [Keep a Changelog](#). Versions follow [Semantic Versioning](#).

Rule. Every change is logged here in the same pass as the code, together with the matching updates to the white paper, the deck, and the technical documentation. A prior conclusion is never silently rewritten; what changed and why is stated.

[3.0.1] — 2026-07-24

Fixed — two of the twenty-five natures were missing

`CHOMP/engine/champ-model.js` held 23 natures. **Naughty** (+Atk / -SpD) and **Lax** (+Def / -SpD) were absent from the table, and an absent nature falls through to the neutral multiplier. Those sets therefore computed with 1.0 where the game applies 1.1 and 0.9 — a Naughty Kingambit read 187 Attack instead of 205. Both are now present and verified against the Champions calculator.

Added

- **S10 — enumerate the domain, do not spot-check it.** Where a rule has a closed, known domain the test walks every member and asserts the expected behaviour, plus a count assertion that the reference list is complete.
- `tests/test-mega-and-boosts.js` now iterates **all 25 natures** and asserts the *direction* of change for all five stats against a neutral baseline, and pins the Champions stat formula (`stat = (base + 20 + SP) x nature`, `HP = base + 75 + SP`). 24 -> 28 assertions.

Notes

- The earlier nature check reported "unexpected multipliers: none" and was **wrong to be reassuring**. It asked whether any nature produced a multiplier outside {0.9, 1.0, 1.1}; a missing nature produces 1.0, which is inside that set. A missing row and a legitimately neutral row are indistinguishable by count, which is exactly why S10 asserts direction instead.
 - `docs/ARCHITECTURE.md` -> v1.1: fault 1.9 and standard S10 recorded.
-

[3.0.0] — 2026-07-24

Architecture — the plumbing, reviewed and standardised

`docs/ARCHITECTURE.md` is new: a blunt review of the whole system, ten engineering standards drawn from it, and the check that enforces each. Every fault named there is one this project actually shipped. The pattern behind all of them was singular: **knowledge with more than one home, and no mechanism that noticed when the homes disagreed.**

Fixed — faults found by the review

- **Three implementations of the same rules** (canonical engine, browser rollout, embedded site copy). When the canonical engine learned real mega base stats the rollout was left behind and the two disagreed by **30%** on Charizard-Mega-Y's Special Attack, silently. Fowler's Rule of Three says the third duplication is the moment to act; we were past it.
- **`data/engine-data.js` stored only DERIVED values** (level-50 stat lines, no base stats). Data that cannot be recomputed can only be copied — which is exactly why the browser engine could not follow the forme fix. It is now generated by `build/build_engine_data.js` and carries base stats.
- **A mega dex merged into the wrong artifact.** 67 formes were added to `engine-data.js` and reported as "in the engine dex"; the damage engine reads a different file entirely, so the fix reached nothing. The claim was made in good faith and was false.
- **Two hand-maintained mega-ability tables** from different sources (Showdown and Serebii). They agreed by luck. Now one generated file, `data/mega-formes.json`, read by both.
- **Identifiers were not normalised across boundaries** (`"sand stream"` vs `"sandstream"`).

Added

- **`CHOMP/tests/test-engine-contract.js`** — a consumer-driven contract test (the Pact pattern): an executable statement of what every implementation must agree on. It caught the mega drift on its first run. 20 assertions.
- **`CHOMP/data/move-effects.json`** — one secondary-effect rulebook generated from Showdown's `moves.json`: 954 moves, 211 with a secondary effect, 33 that can flinch, with accuracy, status chances and stat drops. Two rules that are easy to get wrong are stated once and tested: **a flinch only lands if the user moves first** and expires at end of turn; **a status cannot apply to a Pokémon that already has one**, and type immunities hold (Scald cannot burn a Fire type — verified at 0%).

- **Store-shape invariants** in `sanity_check.py` (S7): `brought ≤ six`, `lead ≤ brought`, winner is a player, every field present, nobody brings more than four. A parser change that breaks the store now fails immediately — the mega double-count silently collapsed CHOMP-EV's eval set from ~1,200 games to **43** before anything noticed.
- **Open team sheets are now parsed.** `|showteam|` declares every Pokémon's item, ability, all four moves and nature — the entire hidden-information problem removed. We were detecting these lines only to set a flag and discarding the sets. 1,624 Bo3 games are affected; that data now accumulates.
- **Forfeit flag** captured at parse time (1,911 of 7,948 games, 24%), so quality filters no longer need the raw archive.

Changed

- Mega formes: all **95 stones** now carry real base stats, typing and ability. Pre-mega state is explicit and overridable (`premega: true`) — holding the stone is not the same as having used it.
- Stat stages (± 6) are supported for attacker and defender, with the full crit rule pinned: a crit **ignores** the attacker's drops and the defender's boosts, and **keeps** the attacker's boosts and the defender's drops. Only the first of those four was previously asserted.

Notes

- The Champions stat formula simplifies exactly: **normal stat** = **(base + 20 + SP) × nature**, and **HP** = **base + 75 + SP**, for every base value 1–255. SP budget is 66.
- Largest remaining gap, stated plainly: the canonical engine's dex is still scraped from an HTML file and parsed with `eval()`. That is the reason the wrong-artifact fault was possible, and it is the top of the remaining list in `ARCHITECTURE.md` §5.

[2.10.1] — 2026-07-24

Changed — every model re-run on the deduplicated store

All reports were still computed on the 14,361-line store that turned out to be ~half duplicates. Re-run on the 7,315 unique games. What survived, and what did not:

Model	On clean data	Verdict
PORY (mid-game value)		holds

Model	On clean data	Verdict
	log-loss 0.5648 vs coin 0.6931, ECE 1.8%, CI [0.550, 0.584]	
WAR (species RAPM)	0.6856 vs coin 0.6931, accuracy 54.1%	holds
MEDICHAM damage	100% within 5% of the Smogon calculator	holds
SLOWKING (species)	greedy 0.078 vs Nash 0.0002, gap CI [0.0031, 0.0977]	holds
SLOWKING (playstyle)	gap CI [-0.0001, 0.2105]	no longer clears zero
COUNTERPLAY tech-lift	+0.0274, CI [0.0006, 0.0533] (was +0.0386 [0.0155, 0.0617])	barely holds
GURU predictive test	log-loss 0.7007 vs coin 0.6931	still a coin, as always documented
CHOMP-EV	winners lean more aligned, CI includes 0.5	still the honest null
XATU policy clone	top-1 32%, phase-conditioning again did not help	unchanged

- The pattern is consistent and worth stating plainly: **the strong results were unaffected and the marginal ones got weaker**. Duplicated rows inflate apparent independent evidence, so they narrow intervals without adding information. Nothing that was solid became shaky; two things that were already borderline (the playstyle equilibrium gap, the tech-lift) lost the significance they had been credited with.
- Site data regenerated from the clean store (7,315 games, 48,326 turns, 11 archetypes).

[2.10.0] — 2026-07-24

Fixed — the store was half duplicates, and every sample size was overstated

- `data/games.ladder.jsonl` held 14,361 lines but only 7,315 unique games — 7,040 duplicate rows, left over from the `merge -X ours` reconciliations during the earlier git incident. Every "14,355 games" figure quoted today was inflated roughly 2x, and duplicated rows also narrow confidence intervals artificially. The store is now deduplicated; **no unique game was lost** (verified by id set comparison against the pre-merge backup).
- **Store reparsed from the raw log archive**, so the mega/weather/terrain parsing now applies to history rather than only to new games. Effect: **12,146 mega events** and **8,752 weather/terrain**

events where there were none, and setter abilities appear at last — Pelipper **Drizzle 0** -> **1,490**, Torkoal **Drought 0** -> **613**, Incineroar Intimidate 1,934.

- **Illegal ability readings rejected.** Log attribution is imperfect (Trace copies opponents; a mis-attributed slot handed Basculegion an "Intimidate" it can never have). Observed abilities are now validated against the species' legal set from the dex, and impossible readings are dropped instead of creating phantom roles.

Changed — a result got weaker on clean data, and that is reported

- **COUNTERPLAY's tech-lift, recomputed on the deduplicated store: +0.0274, 95% CI [0.0006, 0.0533].** It was +0.0386, CI [0.0155, 0.0617] on the duplicated store. The interval still excludes zero, but only barely — the earlier version overstated the evidence because duplicate games were counted as independent observations. The direction stands; the confidence does not. **WAR is unaffected in direction and still beats a coin: 0.6856 vs 0.6931, accuracy 54.1%.**

Notes — what "known" actually means under closed sheets

- A correction to how the new `species-abilities.json` was described. **Nothing about an opponent's Pokemon is known until it is proven in play — ability, item, and moves alike.** The dex narrows the *possibilities*; it does not reveal the set:
- **94 species have exactly one legal ability**, so for those the species genuinely does determine it at preview. That is the only truly certain case.
- **213 species have two or three**, so the ability stays a belief until the log proves it — Basculegion is Swift Swim *or* Adaptability *or* Mold Breaker.
- **A mega's ability is certain only once it megas.** Before that you cannot see the stone, so you do not even know a mega is coming, let alone which form.
- **Items and moves are never given by the dex at all** — only revealed by use.
- **EVs are the hardest of all, and are never stated anywhere.** They are only *bounded*, and only by inference: a damage roll narrows an attacking stat to a range rather than a value (16 possible rolls, all consistent with a band of EVs), and moving first only proves a Speed *inequality* against whatever it outsped — sharpest at a known benchmark, useless in Trick Room or under a Choice Scarf. So an EV spread never collapses to a point the way an ability or an item does; it narrows to an interval that tightens with every turn. The engine's level-50 numbers assume a standard competitive spread and are labelled an approximation for exactly this reason.

- The consequence is a design one: this is a belief-state problem, not a lookup. The right structure is a per-slot information state that starts as the legal possibility set and collapses on each reveal. That is XATU's job, and it is the next thing to build properly.

[2.9.0] — 2026-07-24

Fixed — the extractor never knew mega evolution existed

- **engine/durable-ingest.js** did not parse `|detailschange|` or `| -mega|`. Every mega therefore kept its BASE form's identity, and its new ability was never attributed to anything: **904 of 906** Charizard-Mega-Y sets had a blank ability, and Raichu-Mega-X (Electric Surge) was indistinguishable from Raichu-Mega-Y (No Guard) even though they play nothing alike. Now parsed, with the mega, its base form and the stone recorded.
- **Weather and terrain setters were half-invisible.** A setter ability is usually stated *only* in `| -weather| ...[from] ability: Drizzle|[of] plb: Pelipper`, which we did not read. Verified on a live replay: the fix recovers `pelipper -> Drizzle` and `glimmoramega + Glimmoranite`.
- **engine/roles.py** had stopped finishing (>120 s at 14k games). Two hot spots, both replaced with the identical calculation vectorised: the logistic fit, and a bootstrap that was re-running the model 600 x `|test|` x `|roles|` times when per-row losses do not change between resamples. **5.2 s** now.

Added

- **Mega dex from the authoritative source** (`engine/build_mega_dex.js -> data/mega-dex-official.json`, merged by `engine/merge_mega_into_engine.js`). Source is Showdown's own `pokedex.json` — the data the server runs this format on. **67 mega formes** added to the engine dex with real types, abilities, base stats and required stone. Damage validation re-run and unchanged (100% within 5%).
- **Zoroark-Hisui illusion detector** (`engine/illusion.js -> data/illusion.json`). Illusion copies the NAME, not the moveset, so a legality contradiction proves the disguise: the apparent species cannot learn the move and Zoroark can. On 395 Zoroark team-sides it proves **156 disguises** (0.39 each). Most common disguise Whimsicott; the giveaway moves are Hyper Voice (32) and **Bitter Malice (30)**, a Zoroark-Hisui signature. Conservative by construction, so the count is a floor.

- **Weather and terrain roles split by type** — rain/sun/sand/snow and psychic/grassy/electric/misty, on BOTH the setter and abuser side, so "Swift Swim with no Drizzle" is a detectable defect rather than two generic tags that never meet. Taxonomy is **52 roles**, covering **98.0%** of real move usage.

Notes — measured, including what it cannot yet measure

- **Mega abilities cannot be harvested from logs, at all.** Mega evolution emits only `detailschange` and `-mega`; no ability line follows. An earlier harvest appeared to find 9 conflicts with the official dex — three were **Trace** correctly copying an opponent's ability, and the rest were attribution noise on 1–6 observations. The official dex is the source; the harvester is kept only to discover which formes exist.
- **First dead-ability measurement:** 72% of teams carrying an Expanding Force user have **no Psychic Terrain setter**, and 99.6% of Electric Terrain abusers have no setter. The weather equivalents return nothing yet — those roles are ability-based, abilities only announce sometimes, and the Wilson credibility gate correctly drops them as under-observed. Fix is to source abilities from the dex (certain where a species has only one) rather than from observation; not done yet.
- The store still needs a **reparse** before any of the new mega/weather events exist historically.

[2.8.1] — 2026-07-24

Notes — a wrong diagnosis, corrected

- The `auto: <date>` commits were diagnosed as a rogue timer running `push-all.bat` every ~2 minutes. **That was wrong.** Reading the commits properly showed they are authored by the workspace's own auto-commit, which fires ~2 minutes *after files change*, then commits and pushes. The apparent fixed cadence was simply a long stretch of continuous editing. Verified: a test edit was committed and pushed to origin unattended, with nothing outstanding afterwards.
- Consequence: the publish automation the project needed **already existed**. The idle-publisher scripts added earlier today (`build/auto-push.ps1` , `AUTO-PUSH-START.bat` , `AUTO-PUSH-INSTALL.bat` , `find-autocommit-task.bat`) were removed — a second publisher racing the first is what wedged the repo mid-rebase in the first place. One publisher is correct; two is a bug.

Kept

- `push-all.bat` stays disarmed (requires the `G0` argument) and now refuses to act on a repo that is mid-rebase. That guard is the durable fix and is unrelated to the misdiagnosis.

[2.8.0] — 2026-07-24

Fixed

- `engine/guru.py` **aborted on a truncated store line**, so the site's headline numbers were frozen at **5,199 games** while the store had grown past **7,400**. The ingest job appends on a schedule and an interrupted run can leave a partial line (6 of 7,449 lines). GURU now skips unparseable lines like the other engines. Map metrics are live again.
- `engine/vocab.py` **ignored the multi-role override table**, so tagged moves (Psychic Fangs, Brick Break, Stockpile) were reported as untagged. Coverage of real in-battle move usage is **97.1%**.
- **The MAP's hover text** was long and carried leftover nickname glosses ("(MEDIcham)"). Every node blurb rewritten to one plain sentence.

Changed

- **MAP edges are curved rather than straight**. A crossing-free drawing is not possible for this dependency graph — ALAKAZAM has six parents spread across a row and MEDI feeds both the far left and the far right — so instead of claiming otherwise, each edge now bows in proportion to how far it travels, which separates lines that previously overlapped. Hover still isolates one path.
- **Taxonomy expanded 27 → 39 roles**, all from real gameplay distinctions: status split by type (**burn** = a debuff that halves Attack, **paralysis** = speed control, **sleep** = action denial, **poison** = a clock) — each 1,000+ uses and 20–59 species, so none is sparse; **spread attacker** split from **field-wide (hits your own partner)**, because Earthquake / Surf / Discharge constrain team building (they need an immune partner); plus **multi-hit** (breaks Sash, Sturdy, Multiscale, Substitute), **fixed/fractional damage** (Super Fang, Seismic Toss — ignores stats and the type chart), **residual chip**, **hazards** (incl. Toxic Debris), **substitute**, **weather/effect denial** (Cloud Nine, Air Lock, screen-breakers), **weather/field abuser** (distinct from setter), and **positioning** (now including Roar-family forced switches). Freeze is documented as functionally sleep but *not* tagged: no move in Reg M-B sets it, so crediting every Ice attack would be dishonest.

- Ability mapping corrected throughout: Lightning Rod / Storm Drain → redirection; Hospitality and Regenerator → healing; trigger-boosters (Defiant, Competitive, Moxie) → setup. Fairy Aura and Dark Aura are deliberately **left untagged** — they are passive, permanent, type-wide multipliers, not the same job as Helping Hand's active one-turn boost.

Added

- `engine/build_roles_js.py` → `data/roles.js`; the Roles booth gains a **species explorer** showing any Pokémon's role *distribution* with confidence intervals, and archetypes now show their most distinctive Pokémon by lift.

Notes — the fragmentation trade-off, stated plainly

- A finer taxonomy costs cell size. The median role-pair cell has moved **7,971** (over-tagged, v2.6) → **95** (credible tags, 27 roles) → **~50** (39 roles). It is still far above the old single-label archetype cells (n=11–18), but the direction is the price of resolution. The test and sanity bars were lowered 100 → 50 → 35 **with the reason recorded in-line**, and they now act as the tripwire against adding roles without a justification.
- Winner-prediction from preview roles is unchanged at the coin (0.6935 vs 0.6931); WAR still beats it (0.6867, accuracy 54.4%). Neither headline conclusion moved.

[2.7.0] — 2026-07-24

Changed

- **A species now has a role DISTRIBUTION, not a role list.** The same species is support on one set and offensive on another, so `dex[species][role]` is now `p(role | that species appears)` measured across its revealed sets, and a team's role vector is a **noisy-OR** over its six ($1 - \prod(1 - p_i)$ = probability at least one of them plays the role). Under closed sheets this is also the correct object: before the set is revealed, the distribution is our belief.
- **Credibility is judged by a Wilson lower bound on the rate, not a flat count.** The old `count ≥ 2` rule could not tell a real minor set from noise — it tagged Basculegion as *debuff* on **2 of 3,566** appearances (0.06%). A role now counts only when the Wilson lower bound of its per-set rate clears 5%, which automatically demands more evidence from a common species and stays honest for a rare one.

Fixed

- **Ability → role gaps found by audit:** *Lightning Rod* and *Storm Drain* now map to **redirection** (they redirect, and were untagged). Added a new role **weather/field abuser** (Chlorophyll, Swift Swim, Sand Rush, Solar Power, Protosynthesis, Quark Drive...) — a distinct job from *setting* the weather. Trigger-boosters (Defiant, Competitive, Moxie, Justified, Berserk...) now map to **setup**. Taxonomy is 27 roles; curated roles cover **90.8%** of real in-battle move usage.

Added

- **docs/ROLE-ATLAS.md (+ PDF)** — the master list: every move (478) and ability (99) with the role it is tagged as, generated directly from `engine/roles.py` so it cannot drift, plus a ranked list of untagged-but-used moves as tagging candidates. Generator: `engine/role_atlas.py`.
- **docs/ROLE-FAMILY.md (+ PDF)** — one read-through of the role model, WAR, and the emergent NMF archetypes, with every result stated against its baseline.
- Site: the MAP tab now renders a real **map icon** in the nav, town and room hero (the Porygon2 sprite is gone), and **MAP is first** in the nav order.

Notes — a prior number changed, and why (not silently rewritten)

- The v2.6.0 claim "median role-pair cell **n = 7,971**" was **inflated by over-tagging**: under the old binary rule a team carried **19.6 of 26** roles on average, so nearly every game landed in nearly every cell. With credible tags a team carries **4.3 of 27**, and the honest median cell is **n ≈ 95**. That is still far above the old single-label archetype cells (n = 11–18) — the pooling argument stands — but the earlier figure overstated it. Test and sanity bars moved 100 → 50 with the reason recorded in-line.
- Winner-prediction from preview roles remains at the coin (0.694 vs 0.693); WAR still beats it (0.6875). Neither conclusion changed.
- The hourly **ingest** workflow no longer fails the run when Showdown rate-limits or serves a bad log (`continue-on-error` on the fetch/rebuild steps) — it was emailing a failure notice every hour.

[2.6.0] — 2026-07-24

Added

- **Emergent roles via NMF** (`engine/nmf_roles.py` → `data/nmf-roles.json` , `data/nmf.js`). Instead of hand-declaring roles, this factorizes the data (Lee & Seung 1999; topic-model / Label Distribution Learning framing, Geng 2016) into roles that are *discovered*. Two cuts: (1) team×MOVE usage → offensive cores (recon-err 0.79; attacking moves dominate — shown honestly); (2) team×ROLE → **emergent archetypes** (recon-err **0.53**, the clean view): Intimidate+Fake-Out control, physical offense, special offense+sustain, **bulky wall + screens + redirection**, Tailwind+Encore, priority. A team is a *blend* of these, never one hard label — the structural fix for the single-label grid.
- **Vocabulary census** (`engine/vocab.py` → `data/vocab-usage.json`): tags every move/ability/item and counts **actual in-battle usage** (from the turn log), not just sheet reveals. Curated roles cover 90.4% of non-neutral battle usage; surfaces high-usage uncovered moves as tagging candidates.
- **Roles booth on the site** (`web/index.html` → `app/`): the "Role Foundry" (Smeargle) renders the emergent archetypes and offensive cores live from `data/nmf.js` , with the reconstruction errors shown.
- Role taxonomy grown to **26** roles (added ally-support/positioning and item-disruption); factual multi-role membership for multi-effect moves (Matcha Gotcha = attack+heal+status; Body Press = wall+attack; Knock Off = attack+item-strip; Fake Out = tempo only, not an attacker).

Changed

- **Removed hand-set role weights**. An earlier draft assigned fractional primary/secondary weights (0.6, 0.4...) by hand — asserted, not measured. These were stripped: role *presence* is binary and data-justified, and graded *strength* is now the learned output of the NMF, not a typed input. This is the project's "measured, not asserted" rule applied to itself.
- Sanity extended to 77 checks (role model + WAR + NMF); `tests/test-roles.py` at 19 checks.

Notes

- Honest read on the NMF: at the team level the dominant axis of variation is offensive core + speed control, so the move-level cut is coarse; the role-level cut is the useful one. Rank and the human names are the only non-data choices. Rigorous rank/weighting selection by topic

coherence (Mimno 2011) is noted as the next refinement — reconstruction error alone is not comparable across weightings.

- Still local + (about to) push. Site booth added; white paper / deck / technical docs for the role family remain to be written in a dedicated docs pass.

[2.5.0] — 2026-07-24

Added

- **ROLE model — multi-label team composition** (`engine/roles.py` → `data/pokemon-roles.json`, `data/role-matchups.json`, `data/roles-eval.json`). Replaces the single-label playstyle view. A team is tagged with every role it reveals (24 roles across speed control, weather, terrain, disruption, status/debuff, priority, prankster, setup, healing, screens, walls, pivot, trapping, perish, and physical/special attacker), where each **species earns a role from data** — it is credited once observed doing it (≥ 2 times). Team role vectors are built from the **team-preview six** (leak-free). Why: the old model forced one label per team and shattered the data into archetype $\times$ archetype cells of $n=11-18$; role-pair pooling gives a **median cell of $n=7,750$** (576 cells).
- **Role-pair matchup matrix** with Wilson CIs — the descriptive "which role beats which," now dense enough to reach significance, ready to feed GURU/KING and the site grid.
- **Win-credit attribution:** per-role logistic coefficients (each role's marginal contribution to winning) plus **KO-credit per species** from the turn log (who actually scored the knockouts in games their side won).
- **WAR for Pokémon — Wins Above Replacement** (`engine/war.py` → `data/war.json`). Ridge-regularized Adjusted Plus-Minus (basketball RAPM) on team-preview species indicators, with an explicit 20th-percentile replacement baseline and the logistic wins conversion ($0.25 \cdot \Delta\beta \cdot \text{games}$).
- **Tests + sanity:** `tests/test-roles.py` (19 checks; hand-derived tags, reads shipped reports so it can't drift). `engine/sanity_check.py` extended to cover the role model + WAR (now 70 checks).

Notes

- **Two honest results, stated plainly.** (1) Predicting the winner from **preview roles** ties a coin (held-out log-loss 0.6938 vs 0.6931) — consistent with the sheet-level null; the role model's value is descriptive + attribution, not prediction. (2) But the **species-level WAR model does**

beat a coin ($0.6875 < 0.6931$) and beats the rating baseline (0.6905): *which* species you bring at preview carries a small real signal that raw roles and raw sheets do not. Effect sizes are small; WAR magnitudes are ridge-shrunk and flagged exploratory.

- White paper / deck / technical docs and the site grid are **not yet** updated for this model — code and reports are written and tested locally; the doc + site pass is the next step (flagged, not silently skipped).

[2.4.0] — 2026-07-23

Added

- **v2 models on the site** (`web/index.html` → `app/`): new booths for **GURU** (meta matchup matrix), **XATU** (opponent belief), **PORY** (mid-game win% — with a live interactive "your win %" demo driven by `data/pory.js`), and **ALAKAZAM** (the in-battle capstone, honestly flagged in-development). SLOWKING's booth rewritten to show its real equilibrium mixture + a **rock-paper-scissors triangle diagram**. Model names rendered in **ALL CAPS** across the nav and town.
- **Playstyle layer** (`engine/playstyle.js` → `data/playstyle-matchups.json` ; SLOWKING re-run → `data/slowking-playstyle-eval.json`): classifies each real team by playstyle and builds a playstyle×playstyle matrix; surfaces the strongest non-transitive cycle (TrickRoom → HyperOffense → Sand → TrickRoom).
- **ALAKAZAM plain-English one-pager** (`docs/ALAKAZAM-one-pager.md` + `.pdf`): context, what it will be, how it works, compute needs, timeline — for a non-Pokémon audience.

Changed

- **Honest framing of the playstyle cycle**: the cycle legs are 62% / 71% / 67% but on only $n=13-18$ games each, with 95% CIs that cross 50%. Site copy now calls it a **suggestive pattern, not a settled fact** — it sharpens as the store grows. No overclaiming.
- **Docs folder cleaned**: 13 superseded/duplicate files (old simulator whitepaper, special-cut, v2-plan PDF, old summaries/reviews/handoffs) moved to `docs/archive/` ; one canonical version of each kept.
- **Site chrome**: removed the static side-advisor mascot (kept the roaming Abra sprites).

Documentation (full v2 rewrite — brought to standard)

- **White paper, deck, and technical docs rewritten for v2** (they had drifted to pre-pivot v1). The white paper now covers the empirical ceiling, every model with its validated result (incl. the two honest negatives), the mathematics (Wilson interval, value-net logistic, regret-matching/exploitability, clustered + Beta-resampled CIs, HodgeRank for future core analysis), limits, and cited sources. The deck is plain-English; the technical docs are ASD-STE100 Simplified Technical English organised by Diátaxis. Each ships a matching **PDF**.
- **New docs/SUMMARY.md (+ PDF):** one-page whole-project + per-component summary table.
- **Corrected an error** in the ALAKAZAM one-pager: poker is *sequential*, not simultaneous — reframed as "hidden information like poker **plus** same-time choices like rock-paper-scissors."
- **CLAUDE.md** now lists the docs that **MUST** update in the same pass as any change (living-docs enforcement), so the white paper/deck/technical docs cannot silently drift again.

Removed

- Nothing deleted — superseded docs are archived (reversible), not destroyed.
-

[2.3.0] — 2026-07-23

Added

- **SLOWKING preview-Nash** (`engine/slowking_preview.py` → `data/slowking-eval.json`, `data/slowking.js`): solves GURU's real 13-archetype matchup matrix (5,199 games) to an equilibrium mixed strategy and grades it by **exploitability** (the spec's acceptance bar for the strategic layer) against greedy-single-deck and uniform baselines, with a bootstrap CI that propagates matchup-count uncertainty (Beta resampling). Also reports the strongest **non-transitive 3-cycle** in the meta.
- **Playstyle layer** (`engine/playstyle.js` → `data/playstyle-matchups.json`; SLOWKING re-run with `MATRIX_FILE=.../playstyle TAG=playstyle` → `data/slowking-playstyle-eval.json`): a rule-based classifier tags each real team by playstyle (TrickRoom / Rain / Sun / Sand / Snow / Setup / PerishTrap / TailwindOffense / FakeOutBalance / Stall / HyperOffense) and builds a playstyle×playstyle matrix from 2,866 games. **This is where the non-transitivity is real:** greedy single-playstyle exploitability 0.115 vs Nash 0.0002, gap CI [0.001, 0.280] (clears 0), with a clean cycle **TrickRoom** →

HyperOffense → **Sand** → **TrickRoom** (~0.115 edge/leg). Equilibrium: Rain 0.51 / Sand 0.26 / HyperOffense 0.10 / Setup / PerishTrap / Snow.

- **Test + CI:** `tests/test-slowking.py` — a hand-derived Rock-Paper-Scissors unit test of the Nash solver (answer is uniform, value 0) plus shipped-artifact invariants; gated in the `tests` workflow (regenerates the artifact then checks it).
- **Portfolio:** ABRA added to `willhoop.github.io` as a one-object entry in the `PROJECTS` array (its own convention), leading with a measured number (PORY 0.567 vs coin 0.693). A `PUSH-T0-GITHUB.bat` was added to the portfolio repo for one-click publishing.

Findings (honest)

- **Equilibrium mixture:** Kingambit-Basculegion 0.84 / Garchomp-Incineroar 0.16. Exploitability **Nash** $\approx$ **0 vs uniform 0.109** — mixing over the right decks is far less exploitable than spreading evenly.
- **Greedy $\approx$ Nash at the archetype level:** this meta is currently near-transitive (a dominant deck), so "pick the single best deck" is about as unexploitable as the equilibrium *right now* — stated plainly rather than spun as a win for mixing. **However** a real rock-paper-scissors cycle exists (Charizard-Venusaur → Whimsicott-Garchomp → Garchomp-Incineroar → back, ~0.10 edge/leg), and the greedy-vs-Nash gap CI reaches 0.27, so under plausible resamples the meta is non-transitive. Finer, playstyle-level archetypes (stall / Trick Room / perish-trap / setup) would expose more cycles — the documented next refinement.

Notes

- Archetype-level, not set-level: SLOWKING solves over 13 discovered archetypes, not exact teams/sets; a belief over the opponent's real six (XATU) is the next refinement. Exploitability grades the preview *decision*, never who wins a match (GURU's own predictive test ties a coin).

[2.2.0] — 2026-07-23

The v2 decision-stack release: stop predicting winners, support decisions. Models built + graded this session, each with a proper score + clustered-by-game CI + honest baseline, persisted to JSON.

Added

- **GURU** — meta/matchup matrix from REAL game outcomes with Wilson CIs (`engine/guru.py` → `data/guru.js`). Replaces the biased *simulated* payoff matrix at the source.

- **XATU** — opponent belief (item/ability/moves) inferred from replays (`engine/xatu.py` → `data/xatu.js`).
- **PORY** — mid-game win-prob value net from real replays (`engine/pory.py` → `data/pory.js`). **The win:** held-out log-loss **0.567 vs coin 0.693**, beats a material-sign heuristic, calibrated (ECE 1.6%), clustered-by-game CI [0.548, 0.583]. Proves the v2 pivot — mid-game state is predictable even though pre-game sheets are not.
- **PORY wired into KADABRA:** the coach now shows a per-turn "**you're at X%**" chip at each key moment, computed in-browser from `data/pory.js` (site includes it now). `web/index.html` `kadBuild / renderKad` ; mirrored to `app/` .
- **CHOMP-EV proof** (`engine/chomp_ev.js` → `data/chomp-ev.json`): the winnable team-preview test — do CHOMP's recommended brings beat humans' actual brings on held-out games? Ranks each side's actual bring among all 15 candidate brings by CHOMP exact-damage coverage; headline sign test + held-out logistic log-loss (Brier too), clustered bootstrap CI, baselines = coin / Elo / usage-prior, plus a forfeit-robustness pass and a measured selection audit.
- **Test + CI:** `tests/test-chomp-ev.js` validates the committed `data/chomp-ev.json` invariants (split bookkeeping, score ranges, CI brackets, verdict-vs-numbers consistency, honesty block); gated in the `tests` workflow.

Findings (honest)

- **CHOMP-EV is a NULL at the format ceiling.** On 1,205 held-out human games, CHOMP's bring ranking does **not** beat a coin (log-loss 0.6918 vs 0.6931, CIs overlap), ties an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (0.512, CI [0.493, 0.535] — includes 0.5). CHOMP's top pick matches the human bring ~9.5% of the time (chance 6.7%). **Robust:** dropping all forfeits leaves it unchanged (0.505; log-loss 0.690 vs 0.693). **Selection audit:** eval games average 6.5 turns / 1280 rating vs 6.08 / 1267 excluded — a mild bias that, if anything, *favors* CHOMP, making the null conservative.
- **What this does NOT impugn:** CHOMP's damage math stays **VALIDATED** vs `@smogon/calc` ; the null is about the *bring-selection signal*, which sits at the same near-coin ceiling as pre-game win prediction. It guards against optimizing a bring metric with no held-out winning signal (the DITTO/measure-gaming trap). Path to a real edge: score brings with belief-aware value (XATU) + the lead stage-game (SLOWKING) + PORY leaf value, then re-run this exact test.

Notes

- White paper and plain-English deck updates for GURU/XATU/PORY/CHOMP-EV are still pending (this pass shipped code + CHANGELOG + MODELS/HANDOFF; the long-form docs are the next documentation pass).
-

[2.1.0] — 2026-07-23

Validated

- **MEDICHAM damage engine validated against @smogon/calculator (engine/validate_damage.js)**: with stats aligned, matches the ground-truth calc within 5% on 100% of 31 meta scenarios (median 0% error). Fixed the level-50 harness bug, then closed the ability gaps it surfaced.

Added

- **Ability/item layer, each validated vs Smogon:** Ruin quartet, Solar Power, Guts, Orichalcum Pulse, Hadron Engine, Adaptability, Technician, Tinted Lens, Filter/Solid Rock, Multiscale, Thick Fat, Heatproof, Purifying Salt, type-immunity abilities, Expert Belt, Muscle Band, Wise Glasses.
- **DITTO policy hardening:** accuracy-weighted move value, recoil cost, self-stat-drop moves (Close Combat/Superpower/Overheat) with **Contrary** flip, **Mega ability tracking** (base vs Mega stone — Staraptor → Contrary, Swampert → Swift Swim + canonical Megas), weather-speed abilities (Swift Swim/Chlorophyll/Sand Rush/Slush Rush). Reduces the speed/frailty over-crediting (the Staraptor problem).
- **Site now grows:** `data/live.js` (counts + data-derived archetypes) and `data/kad-replays.js` (offline replay bundle) regenerate via `engine/refresh-site-data.py`, run by the daily replay pull. Town stats + DITTO archetypes read live.
- **Archetypes discovered from data** (`engine/archetypes.py`, k-means over 9,998+ real teams), not hand-listed — refreshes as the meta shifts.
- **ORB (CHOMP dock) upgraded to a validated Smogon-grade substitute:** reads live stats/items/boosts/weather/terrain/Helping Hand/spread/screens, shows applied conditions. One-click install; auto-updates.

Fixed

- KADABRA works offline (`file://`) — coaches from the local bundle, clean move-by-move viewer with arrows and a bold "what you should've done" (dropped the Showdown iframe clutter).
- Abilities corrected from a curated meta map (no more bogus "Pressure"); real move names + spacing; Laplace smoothing so win rates never read 0%/100%.
- Non-transitivity view rebuilt as big whimsical rock-paper-scissors loops (no tiny text). Town card honesty (daily, real counts). Footer/sprite overlap.

Known gaps (not guessed — need confirmation)

- **Champions rule changes vs Gen 9** (sleep, paralysis, specific move changes) are NOT yet modelled — pending the exact format rules.
 - Enemy EVs are assumed (unknowable). Mega **stats/types** not yet swapped (abilities are).
-

[2.0.0] — 2026-07-23

The "honest instrument" release: a real doubles engine, an evaluation layer that grades every probability, the SLOWKING belief-search stack, and the first learned value function (the flywheel's core). Also a strict self-review that reshaped the roadmap.

Added

- **MEDICHAM v3 — real Gen-9 doubles engine** (`engine/medicham2-browser.js` , embedded in the site): replaces the 1v1 OHKO-chain that collapsed to 0%/100%. Damage formula with boosts/spread/crit/rolls, weather, Trick Room, Tailwind, priority, Protect, items, abilities, Fake Out; behaviour-cloned policy (samples real move rates), need-based Protect. Verified: mirror 0.50, healthy distribution, 400 rollouts/29ms. Win rates now carry a 95% CI on the site.
- **Evaluation harness** (`engine/eval_harness.py`): temporal held-out log-loss / Brier / calibration vs coin, player-Elo, and usage baselines with bootstrap CIs. **Verdict: JOLTEON ties a coin in log-loss** — demoted from headline predictor to fast prior + baseline; site copy made honest.
- **Calibration** (`engine/calibrate.py`): temperature scaling; the Python JOLTEON was 6× overconfident.

- **Learned in-battle value function** (`engine/train_value.py` → `data/value-net.json`): reconstructs per-turn HP state and regresses the outcome. Beats a coin (log-loss 0.682) and is calibrated — the first genuinely learned, calibrated component, and the leaf evaluator + flywheel core.
- **SLOWKING infrastructure** (`engine/slowking/`): `nash.py` (equilibrium, verified on RPS/2×2), `belief.py` (public-belief-state + Bayesian filter), `ismcts.py` (simultaneous-move regret matching, recovers exact Nash), `game.py` (engine interface), `solver.py` (team-preview Nash + continual re-solve; returns bring *mixes* + win%), `value.py` (loads the learned leaf). All unit-tested.
- **Self-play data pipeline** (`sim/generate-dataset.js`) writing engine games into the store schema — the unlimited, unbiased "more games" path. Scraper default raised 2 → 25 pages (~10× per run).
- **Non-transitivity finding** (`data/nontransitivity.json` , DITTO tab): the meta is rock-paper-scissors (3 robust cycles after noise control) — empirical proof an additive rating can't capture it. Shown with an explicit "preliminary, thin data" caveat.
- **Docs**: `docs/POKER-T0-POKEMON.md` (the founding white paper), `docs/THESIS-REVIEW.md` + `docs/THESIS-REVIEW-v2.md` (strict self-critique with fixes), `docs/COMPETITORS.md` (VGC-Bench et al. and how we refine them), `docs/OVERNIGHT-HANDOFF.md` .
- **Site**: in-browser DITTO (item tuning + PokéPaste export) and KADABRA (client-side replay coach); MEDICHAM/DITTO use the sprite picker; saved-teams in the matchup; ORB opens from Chomp's room; per-room personality mascots; threats table with sample-adjusted Win%, real speed, Games column.

Changed / Honest corrections

- JOLTEON reframed as a fast prior, not an oracle (backtest: ~coin in log-loss).
- White paper corrected: the current SLOWKING search is IS-MCTS/PIMC (strategy fusion), a rung below the ReBeL target — no longer overclaimed.
- Non-transitivity presented as preliminary (approximate engine, small sample), not a settled claim.

Fixed

- MEDICHAM special-move bug (special attackers dealt 0 damage); booth slot regression that broke "Surprise me"; DITTO/KADABRA no longer require a server.

[1.0.0] — 2026-07-22

Added

- **ABRA is born**, split out from CHOMP as its own project: the Automated Battle Replay Analyzer. CHOMP stays the bring-4/lead-2 engine; ABRA is the meta-analysis brain that feeds it.
- **Durable, incremental, no-redo ingest** (`engine/durable-ingest.js`): pulls public Champions Reg M-B replays from the Showdown API (paginated, ~200 logs/sec, concurrent), stores every game raw and tagged — both teams' six, brings, leads, observed moves/items/abilities, result, both ratings, and a bot flag. Appends only new games (dedup by id). Tested on 1,501 real ladder games.
- **Analysis over the store** (`engine/analyze.js`): usage model at any rating cutoff / humans-only, plus a personal split by Showdown username. Writes `data/meta-usage.json` for CHOMP.
- **ME alias list** so a Showdown rename is a one-word edit, never a re-pull.
- `tests/test-parse.js` — 12 hand-derived checks on the replay extractor (teams, leads, brings, observed set fields, bot flag, rating, date).
- Governance: LICENSE (MIT), SECURITY.md, CONTRIBUTING.md, .gitignore, CI workflow.

Validated

- High-ladder filter (humans, 1300+) reveals real signal distinct from the raw ladder — e.g. Kingambit 62% win, Incineroar 65% — confirming the tag-and-filter design earns its keep.

[1.1.0] — 2026-07-22

Changed

- **Reframed to its true scope.** ABRA is documented as the live-data platform whose purpose is to feed a simulator that models games and teams — a self-improving flywheel (collect → simulate → optimise teams → play with CHOMP → auto-ingest enemy teams → improve). CHOMP is one small early consumer, not the point. White paper §8 states the flywheel and the honest built-vs-roadmap status.

[1.2.0] — 2026-07-22

Added

- **Simulator research white paper** (docs/ABRA-simulator-whitepaper.md): an MIT-level treatment of learning a VGC battle simulator from logged replays. Formalises the game (POSG, imperfect info, simultaneous moves), derives three modelling tiers with their estimators and failure modes, frames team optimisation and the self-improving flywheel, and grounds every claim in the 2025 literature (PokéChamp, Metamon, VGC-Bench, ReBeL/ Player-of-Games, Sampled/Gumbel MuZero, offline RL). Names the model family in the CHOMP/ABRA tradition — **JOLTEON** (fastest, win-prob), **MEDICHAM** (Rapidash, rollouts), **SLOWKING** (slow deep learned dynamics), **DITTO** (team optimiser) — speed of the Pokémon matches the cost of the model. Folds in CHOMP's pKO threat scoring as JOLTEON's features and MEDICHAM's dynamics (grey-box modelling).

[1.6.0] — 2026-07-23

Major finding

- **The Champions engine is OPEN, not closed** (docs/OPEN-ENGINE-FINDING.md). Verified by cloning smogon/pokemon-showdown : the exact format [Gen 9 Champions] VGC 2026 Reg M-B (and its Bo3 variant) is in config/formats.ts , backed by a full champions mod (SP system in data/mods/champions/scripts.ts). This overturns the project's founding assumption. SLOWKING no longer needs to *learn* the dynamics — it can query the real engine (ReBeL over a known simulator), and MEDICHAM/DITTO/JOLTEON can use exact rollouts + self-play. SLOWKING white paper §3 corrected; roadmap task added to wire the engine as ABRA's simulator.

Added

- **MEDICHAM runs in the browser** — the damage engine, type chart, sets, and behaviour-clone priors (96KB) are embedded in web/index.html ; the rollout runs client-side (~40ms / 200 rollouts). The "MEDICHAM check" button and Medicham's run panel now work with **no server**. Validated: mirror 0.53, rain-vs-sun 0.19 (matches the Node engine).
- **Combined team+rating predictor** (engine/predictability.py §2.5): the real pre-game ceiling is ~57% — combining team sheets AND player ratings does no better than team alone,

confirming the game is variance-dominated (and that the two ~55% are different axes that corroborate, not the same claim). Predictability study updated with the honest framing.

- **ABRA MCP server** (`mcp/`): exposes the models as tools Claude can call — `abra_win_probability` (JOLTEON), `abra_rollout` (MEDICHAM), `abra_threats`, `abra_species_stats`, `abra_optimize_team` (DITTO), `abra_coach_replay` (KADABRA). Local stdio server; `claude mcp add abra -- node mcp/server.js`.
- **Regulation registry + archive** (`data/regulations.json`, `build/archive-regulation.js`): the active regulation is a one-line config edit; ingest/analysis read it. When a reg ends, `archive-regulation.js` snapshots the store + all models into `data/archive/<id>/` (date-stamped, with a manifest) so previous-regulation data is preserved forever. `--rotate` starts a fresh store.
- **Mega Evolution** added to the game's action model (SLOWKING white paper §2) as a per-turn step.
- **Ditto page rebuilt**: team-builder on top + a live, sortable/searchable **threat rankings table** (usage / bring / lead / win% / speed) from the real stats; static chips removed.
- **Team carries between models** (localStorage), hover-× to remove a Pokémon / clear team, usage- ranked picker, in-browser MEDICHAM, lightning flash on JOLTEON, confetti idle-loop stopped.
- **Omnibus is now robust** (`build/omnibus.py`): always emits a self-contained HTML (SVG embedded directly, no LibreOffice), attempts the PDF only if `OMNIBUS_PDF=1` — reproduces the Special Cut reliably as the docs grow.
- New docs sewn into the Special Cut: predictability study, SLOWKING white paper + roadmap, architecture notes.

[1.5.0] — 2026-07-23

Added

- **Recency weighting (concept-drift decay)** in JOLTEON: every training game is weighted $w = 0.5^{** (age_days / \tau)}$ with a half-life τ (default 30d), so the models track the *live* metagame instead of averaging over stale history. Normalised to mean 1 (L2 scale unchanged); $\tau \rightarrow \infty$ recovers equal weighting. No-op on the current 2-day store (reported honestly); unit-verified on synthetic 90-day data (oldest 0.33, newest 2.14). Same rule applies to the usage model and behaviour-clone. Fully documented in `docs/ARCHITECTURE-NOTES.md`.

- **SLOWKING white paper** (`docs/SLOWKING-whitepaper.md`): the definitive Tier-3 design — offline belief-state search over a *learned grey-box model* of the *closed* Champions engine (residual over CHOMP), simultaneous-move mixed-Nash subgames, warm-started by the behaviour-clone. Grounded in ReBeL, Student of Games, PokéChamp (ICML 2025), Gumbel MuZero, Metamon. Plus a **research roadmap** (`docs/SLOWKING-research-roadmap.md`) turning it into five buildable papers.
- **SLOWKING Paper-1 built** (`engine/game-spec.js`): encodes stored replays into (`state`, `observation`, `action`, `reward`) trajectories — 30,608 real state-transitions with actions and terminal rewards. The offline dataset a Tier-3 solver trains on; a re-parse, never a re-pull.
- **Behaviour-clone + status/field MEDICHAM v2** (`engine/policy.js`, `engine/moves-meta.js`, `engine/medicham.js`): the rollout now samples *what real players click* (Tailwind 34% for Whimsicott, Fake Out 30% for Incineroar, ...) and applies the effects — sleep, burn, paralysis, Tailwind (2× speed), Trick Room (inverted order), setup boosts, Protect. Speed control and setup are now *valued emergently*. Fixed a faint-and-replace symmetry bug (mirror back to ~0.50) and gated support on survival (don't set up into a KO). `tests/test-medicham.js`, `tests/test-dynamics.js`.
- **DITTO ported to Node** (`engine/ditto.js`): the whole live app now runs with **no Python**. JOLTEON scoring reimplemented in JS from the trained weights; **MEDICHAM wired in natively as the finalist re-ranker** (coarse-to-fine: JOLTEON proposes thousands, MEDICHAM decides the finalists). In a real run MEDICHAM overruled JOLTEON — chose the rain team (75.7%) over JOLTEON's tyrannitar pick (68% grounded vs 79.8% JOLTEON). `server.js /api/ditto` now calls Node.
- **Local app + server** (`server.js`, `start.bat`, `app/`): the site is served from `app/` and runs the real engines on the user's machine (MEDICHAM/KADABRA/DITTO via Node, JOLTEON in-page). Lazy, robust Python probe (skips the Windows Store stub) kept only for optional JOLTEON *retraining*. Booth is searchable and usage-ranked; team-builder enlarged; live "MEDICHAM check" button.
- **Multi-format + open-sheet tags, dedup-by-replay-id everywhere** (never double-count a game reviewed or self-uploaded), **per-turn extractor + raw-log archive** (any new field is a re-parse).
- **ABRA WORLD website** with per-model "How X thinks" panels, teleporting-Abra background, usage-ranked sprite picker, PokéPaste input (accepts any species; off-roster treated as neutral).

- `docs/ARCHITECTURE-NOTES.md` : the Python/JS split rationale and the recency-weighting design, in detail.

Changed

- Model labels describe the role (win probability, battle rollouts, team optimiser, replay coach, belief search, bring-4 engine), not the Pokémon name twice.

Queued

- **ORB** — CHOMP's auto-fill mid-game calculator (the Life Orb of the CHOMP family): pulls your six (moves/items/EVs) and the opponent's revealed team from the live battle so there's nothing to type mid-game; opens in its own tab.

[1.4.0] — 2026-07-22

Added — the model family (the simulator, stages 2–3, now has working v1s)

- **Per-turn extraction** (`engine/durable-ingest.js` v2): the extractor now captures a per-turn event stream — move order (→ speed), exact damage % per move, fainted, status, and reveals — on every game. Backfilled onto all 4,999 games: **30,611 turns, 55,336 damaging move events**.
- **Raw-log archive + `MODE=reparsed`** : each raw `.log` is archived (`data/*.raw-logs.jsonl`), so any NEW field is a re-parse, never a re-pull. Proven by backfilling `format / openSheet` onto 4,999 games with **zero network calls**. (Archive is gitignored; the extracted store carries the turns.)
- **Dynamics model** (`engine/dynamics.js` → `data/dynamics.json`): observed speed (who-moves-first, incl. Choice-Scarf hints) for 186 species, and observed damage distributions for 1,170 (attacker, move) pairs. E.g. Garchomp Earthquake mean 57%, Basculegion Wave Crash 62.5%.
- **JOLTEON v2** (`engine/jolteon.py`): win-probability model gains **rarity-aware L2 shrinkage** (a species seen 25× is pulled toward neutral; seen 1000× is trusted — a measure-gaming guard at the model level) plus speed-edge and firepower-edge features from the dynamics model. Honest result: ~55% humans-only held-out vs ~49% coin flip; the dynamics features tie species-only (firepower earns weight +0.30, speed-edge is noise at this scale). Reported straight.

- **MEDICHAM built** (`engine/medicham.js`): Tier-2 Monte-Carlo rollout over CHOMP's exact damage engine. Rain core beats sun core 0.60; mirror 0.51; ~1s / 300 playouts. Sequential-singles v1 (honest scope). `tests/test-medicham.js` .
- **DITTO built** (`engine/ditto.py`): team optimiser using JOLTEON as evaluator against a gauntlet of REAL ladder teams, double-oracle rounds, **usage-weighted threat coverage** (guarantees an answer to high-bring threats like Basculegion, ignores rare ones like Camerupt), and a **bias report** showing where rarity shrinkage suppresses a pick. Surfaces the measure-gaming failure honestly: JOLTEON-optimised "90%" team → MEDICHAM rollouts reveal ~12% → this is *why* Tier-2 vets Tier-1.
- **KADABRA v1 built** (`engine/kadabra.js`): turn-by-turn coach over a replay — reconstructs each scene, gives the speed read + damage read (cross-checked vs the ladder average, flags high/low rolls), draws the lesson, and background-appends the game (the flywheel from the coaching seat).
- **SLOWKING scaffold** (`engine/slowking.py`): Tier-3 interface fixed + data-readiness report; honestly flagged as a research effort, not a trained model.
- **Multi-format + open-sheet tags**: `FORMATS=` env supports collecting other ladders (e.g. the Reg-G best-of-3); every record now carries `format` and `openSheet` (bo3 / agreed open team sheet is a distinct information regime). 42 open-sheet games found in the current store.
- **ABRA WORLD website** (`web/index.html`): a Club-Penguin-styled interactive town — one room per model — with the JOLTEON win-probability model running live client-side (real embedded weights), sprite team pickers, animated odds meter, and links to the rest of the portfolio.
- Tests: `tests/test-medicham.js` , `tests/test-dynamics.js` (all green alongside parse + jolteon).

Changed

- The flywheel's honest status advances: stages 2 (simulate) and 3 (optimise) now have working v1 models (MEDICHAM, DITTO), with the tiered vetting (Tier-1 proposes, Tier-2 checks) demonstrated end-to-end. Tier-3 depth (SLOWKING) remains roadmap.

[1.3.0] — 2026-07-22

Added

- **JOLTEON v1 built** (`engine/jolteon.py`) — the Tier-1 win-probability model, a Bradley–Terry logistic over per-species strengths with a min-sample floor (anti-overfit). Trained on 5,000 real ladder games (temporal split, humans only). **Measured: 56.6% held-out accuracy vs 49.6% baseline**, Brier 0.251 (calibrated) — a real, honest, modest edge from team composition alone, as the domain's variance predicts. Ships a `predict` CLI and `tests/test-jolteon.py` (antisymmetry, mirror=50%, coverage, range). Model saved to `data/jolteon-weights.json`.
- **Full ladder pulled** — the durable store grew from 1,501 to **5,000 games** (incremental, dedup).

Notes

- The first training on 1,501 games did **not** beat the baseline; more data (5,000) and a min-sample floor were what cleared it. Recorded honestly: this is why the flywheel (more games over time) and damage-grounded features (§4.3.1) matter, not species identity alone.